

Technical Implementation Plan: Tokyo Mate 東京通

Feature Branch: 001-tokyo-travel-assistant | Date: 2026-09-12 | Spec: spec.md

Executive Summary (執行摘要)

Tokyo Mate 東京通是專門服務台灣自由行旅客的東京 AI 旅遊助手。核心定位為「東京在地私人導遊 × 台日雙語口譯員 × 東京旅遊知識顧問」。

本計畫依據已批准的 Product Specification (spec.md)、UX/UI Design Handoff (ux-ui-design-handoff.md) 與 Engineering Constitution (.specify/memory/constitution.md) 制定。

架構遵循 **Simplest Sufficient Technology** 原則，前端採用 React + TypeScript + Vite，後端採 Vercel Serverless Functions 做為保護 API Key 的最小必要 Server Boundary。AI 核心採用 OpenAI Responses API (GPT-4o/mini) 搭配 Whisper API (Speech-to-Text) 與 Speech API (Text-to-Speech)；地點服務採用 Google Places API (New) REST endpoint；東京知識庫採用 Repository 內結構化 JSON (src/data/tokyo/*.json) 搭配最簡詮釋資料過濾與 Context Injection。第一版不採用資料庫、向量庫或帳號系統，僅維護當次使用之記憶體/UI 狀態。

Technical Context (技術上下文)

項目	決策 / 技術選型	備註 / 驗證依據
Language/Version	TypeScript 5.x + Node.js 18+	型別安全與現代 ES 語法
Frontend Framework	React 18 + Vite	單頁應用 (SPA)，單一 App Shell，第一版不加入 React Router
State Management	React local state / Context	僅真正跨區域共享狀態使用 Context，無 Redux/Zustand
Server Boundary	Vercel Serverless Functions	/api/assistant.ts, /api/transcribe.ts, /api/speech.ts, /api/places.ts
AI Provider	OpenAI Responses API	OPENAI_MODEL 環境變數控制，處理意圖判斷、雙語翻譯、旅遊問答
Speech-to-Text	MediaRecorder + OpenAI Whisper API	/api/transcribe 接收錄音 Blob 轉文字，提供前端修正
Text-to-Speech	OpenAI Speech API	/api/speech 合成正常與慢速 (0.75x) 語音檔供前端播放
Places Service	Google Places API (New) REST	/api/places 支援經緯度 Nearby Search 與文字地區 Text Search
Knowledge Base	Repo 內結構化 JSON 靜態庫	src/data/tokyo/*.json (areas, transport, food, shopping, culture, emergency)

項目	決策 / 技術選型	備註 / 驗證依據
Retrieval Strategy	Simplest Metadata Filtering	Step 1 Tag/Category 詮釋資料過濾 + Step 2 Top-3~5 Entry Context Injection
Styling	Vanilla CSS / CSS Variables	輕量化響應式 mobile-first 樣式 · 無 Tailwind / Heavy UI library
Testing Stack	Vitest + React Testing Library + jsdom	單元測試與 Component 互動測試
Target Platform	Vercel (Web / Mobile-first installable PWA)	單一 Repository 部署 Vite 前端與 Serverless API ; PWA 提供可安裝 App Shell、受控靜態快取與離線 fallback

Constitution Check (工程憲章審查)

GATE: Must pass before implementation.

- **I. Specification Before Implementation:** 本 Plan 嚴格對齊已批准之 `spec.md` 與 `ux-ui-design-handoff.md`。
- **II. Simplest Sufficient Technology:** 無 Database、無 Vector DB、無 Auth、無 React Router、無 Redux/Zustand、無 Axios、無 Monorepo、無不必要依賴。
- **III. Complexity Discipline:** 複雜度等級為 C | AI-Powered App · Generative AI 為核心體驗，其餘架構保持 Simplest Standard。
- **IV. Technology Stack Stability:** 採用 React + TS + Vite + Vercel Functions + OpenAI + Google Places，完全遵守 Approved Technical Truth。
- **V. Privacy by Minimum Necessary Data:** 經緯度僅在一次性 Request 處理中使用，不寫入 LocalStorage 或 DB；第一版不建立長期聊天歷史。
- **VI. Secrets Never Live in Frontend:** `OPENAI_API_KEY` 與 `GOOGLE_PLACES_API_KEY` 僅在 Serverless Functions 存取，`.env.local` 已 gitignored。
- **VII. AI Freshness and Honesty:** `liveDataStatus` 區分已確認與不確定狀態，不確定的即時資料明確提示使用者。
- **VIII. Translation Quality Over Literal Translation:** 中譯日注重東京在地自然情境語氣；日譯中採用台灣繁體中文，避免大陸用語。
- **IX. Safety-Critical Action First:** 護照遺失、生病、災害等緊急問題觸發 Emergency 模式，優先展示即時動作與緊急聯絡，隱藏無關景點推薦。
- **X. Testability:** 提供 Vitest 自動化測試與完整的 Runnable Validation Journeys 手動測試流程。
- **XI. Maintainability:** 扁平、低抽象層之專案結構，依功能導向模組化。
- **XII. Incremental Implementation:** 劃分 9 大實作 Phase，每一 Phase 遵守 Run → Verify → Fix 循環。
- **XIII. Graceful Failure:** 所有外部 API 失敗與麥克風/GPS 權限拒絕均轉換為使用者友善的 `ProductError` 狀態與下一步建議。
- **XIV. Repository Safety:** 非破壞性專案結構，保留既有 `.git/`、`.github/`、`.specify/`、`.vscode/`、`specs/` 資產。
- **XV. Traditional Chinese Working Language:** 所有計畫、設計與說明文件均使用台灣繁體中文。

Project Structure (專案目錄結構)

```
tokyo-mate/
├── .github/                                # GitHub Workflows & Copilot/SpecKit
├── configuration
├── .specify/                              # SpecKit memory, constitution & scripts
├── api/                                   # Vercel Serverless Functions
│   ├── assistant.ts                      # OpenAI Assistant Orchestration
│   ├── transcribe.ts                    # OpenAI Whisper Audio Transcription
│   ├── speech.ts                        # OpenAI TTS Speech Generation
│   └── places.ts                         # Google Places API (New) Integration
├── public/                               # Static public assets (icons, manifest,
offline fallback)
├── specs/                                # Specifications & Design artifacts
│   └── 001-tokyo-travel-assistant/
│       ├── spec.md                      # Product Truth Specification
│       ├── ux-ui-design-handoff.md
│       ├── plan.md                      # Technical Implementation Plan (This file)
│       ├── research.md                  # Phase 0 Research Decisions
│       ├── data-model.md                # Phase 1 Data Model & Interfaces
│       ├── quickstart.md                # Phase 1 Quickstart Validation Guide
│       ├── contracts/                   # Phase 1 Server API Contracts
│       │   ├── assistant-api.md
│       │   ├── transcribe-api.md
│       │   ├── speech-api.md
│       │   └── places-api.md
│       ├── checklists/
│       └── requirements.md
├── src/                                  # Frontend Application Code
│   ├── components/                      # Shared UI Components (Button, Input, Card,
Modal, AudioPlayer)
│   ├── screens/                          # Top-level Screen Views (HomeScreen,
AssistantScreen, KnowledgeBrowser)
│   ├── features/                         # Domain Feature UI Modules
│   │   ├── translation/                 # TranslationResult, ToneAdjuster
│   │   ├── travel/                     # TravelAnswer, ActionPlan
│   │   ├── nearby/                     # NearbyResults, PlaceCard
│   │   ├── knowledge/                  # KnowledgeCard, CategoryFilter
│   │   ├── emergency/                  # EmergencyAnswerCard
│   │   └── speech/                     # VoiceInputModal, EditableTranscript
│   ├── data/                            # Tokyo Knowledge Base JSON Files
│   │   └── tokyo/
│   │       ├── areas.json              # 18+ Tokyo Areas
│   │       ├── transport.json          # Transport & Transfers
│   │       ├── food.json               # Dining & Etiquette
│   │       ├── shopping.json           # Shopping & Tax Free
│   │       ├── culture.json            # Culture & Manners
│   │       └── emergency.json          # Emergency Contacts & Actions
│   ├── services/                        # Client Services & API Client Helpers
│   └── api.ts                           # Client fetch wrappers for /api/*
```

├── knowledge.ts	# Local Knowledge Retrieval & Filtering
├── geolocation.ts	# Geolocation API Wrapper
├── types/	# Shared TypeScript Types & Interfaces
│ ├── request.ts	# UserRequest, UserLocation
│ ├── assistant.ts	# AssistantResult, SuggestedAction
│ ├── knowledge.ts	# KnowledgeEntry
│ ├── place.ts	# PlaceResult
│ └── error.ts	# ProductError, ErrorCode
├── styles/	# Vanilla CSS / CSS Variables / Global Styles
│ ├── variables.css	
│ ├── global.css	
│ └── components.css	
├── App.tsx	# Main App Shell & State Orchestration
└── main.tsx	# React Entry Point
├── .env.example	# Environment Variables Template
├── .gitignore	# Git Ignore rules
├── package.json	# Dependencies & Scripts
├── tsconfig.json	# TypeScript Config
├── vite.config.ts	# Vite Build & Dev Server Config
└── vitest.config.ts	# Vitest Testing Config

Complexity Tracking (複雜度管制)

項目	理由	替代被拒絕方案與原因
Vercel Serverless Functions (<code>api/*.ts</code>)	保護 <code>OPENAI_API_KEY</code> 與 <code>GOOGLE_PLACES_API_KEY</code>	拒絕前端直接呼叫外部 API (違反 Constitution VI 秘密洩漏風險)。
Local Knowledge JSON (<code>src/data/tokyo/*.json</code>)	完全零託管成本、Git 版本管理，檢索極快	拒絕 Vector Database / Supabase / Pinecone (違反 Simplest Sufficient Technology)。

Implementation Phases (實作階段劃分)

Phase 1: App Shell & Foundation

- 建立非破壞性專案結構、`package.json`、Vite + TS 設定、CSS 變數與基礎 RWD Layout。
- 實作品牌 Header、通用輸入列 (文字/麥克風按鈕) 與 4 大入口卡片 (即時翻譯、問東京、探索附近、東京百科)。
- 建立 PWA App Shell、Web App Manifest、必要圖示與離線 fallback；註冊 service worker，僅快取版本化的 App Shell 與穩定靜態知識內容，不快取 API 回應、精確位置、錄音或長期對話。
- 實作網路狀態提示、需要連線功能的 disabled/error 邊界，以及不阻斷目前任務的新版更新提示。

Phase 2: Server Boundary & 文字 AI 核心

- 實作 `/api/assistant.ts` Serverless Function。
- 整合 OpenAI Structured Outputs，實作 `UserRequest` 轉 `AssistantResult` 的強型別回應。
- 前端實作 AssistantScreen 基礎訊息呈現與 API Client Wrapper (`src/services/api.ts`)。

Phase 3: 中日雙向翻譯體驗

- 增強 AI Prompt 支援語調切換 (預設自然、更禮貌、更口語) 。
- 實作 `TranslationResult` 組件：大字日文顯示、一鍵複製、播放語音與語調按鈕。

Phase 4: Tokyo Knowledge Base & 檢索

- 建立 `src/data/tokyo/` 6 大結構化 JSON 資料檔 (`areas`, `transport`, `food`, `shopping`, `culture`, `emergency`) 。
- 實作 `src/services/knowledge.ts` 元資料比對檢索，並將 Top 相關條目注入至 `/api/assistant.ts` System Context 。
- 實作 `KnowledgeBrowser` 分類瀏覽器。

Phase 5: 即時 Web Data 狀態處理

- 在 `/api/assistant.ts` 中結合 OpenAI Web Search 能力處理時間敏感查詢。
- 實作 `liveDataStatus` 標籤 (`verified`, `uncertain`, `unavailable`)，對於不確定資訊呈現警示與後續步驟。

Phase 6: Speech-to-Text & Text-to-Speech

- 實作前端 `VoiceInputModal` 搭配 `MediaRecorder` API 。
- 實作 `/api/transcribe.ts` (Whisper API) 與可編輯辨識文字框 (`EditableTranscript`) 。
- 實作 `/api/speech.ts` (TTS API) 與前端 Audio Player，支援正常與慢速 (0.75x) 播放。

Phase 7: Explore Nearby 地點探索

- 實作 `/api/places.ts` (Google Places REST API) 。
- 前端 `geolocation.ts` 處理主動授權，拒絕時提供手動地區輸入 fallback 。
- 實作 `NearbyResults` 與 `PlaceCard` 地點卡片。

Phase 8: Emergency Mode & Graceful Error Recovery

- 於 `/api/assistant.ts` 實作緊急意圖偵測，產出 `emergencyGuide` 結構。
- 實作 `EmergencyAnswerCard` (優先顯示獨立動作、隱藏無關景點快捷) 。
- 實作全站統一 `ProductError` 失敗處理 (麥克風拒絕、GPS 拒絕、網路斷線、API 異常) 。

Phase 9: Responsive Polish & Acceptance Criteria Verification

- 完成手機、平板、桌面視圖之排版優化。
- 驗證手機瀏覽器與已安裝 PWA 的直向/橫向、安全區域、離線啟動、網路恢復、更新提示與快取隱私邊界。
- 執行全部單元測試與 `quickstart.md` 手動 validation journeys，確保 100% 滿足 Acceptance Criteria 。